



GOVERNMENT POLYTECHNIC MUZAFFARPUR

(Under Department of Science, Technology & Technical
Education Bihar)



LAB MANUAL

Programming with C

Course Code: 2418101

S. No.	Laboratory Experiment / Practical Titles	Relevant COs Number(s)
1	Develop minimum 3 programs using: (a) Constants, Variables, arithmetic expressions. (b) Develop minimum 3 programs to exhibit the use of increment/decrement operators, data type conversion.	CO1, CO2
2	(a) Write a simple program to convert temperature in Fahrenheit degrees to Centigrade degrees (take input from the user). (b) Write simple programs to calculate the area and perimeter of the rectangle, and the area & circumference of the circle (take input from the user).	CO1, CO2
3	Write program to: (a) Determine whether a given year is a leap year or not. (b) Determine whether a string is palindrome. (c) Find the greatest of the three numbers using conditional operators. (d) Find if a given character is a vowel (use an if-else ladder).	CO1, CO2, CO3
4	Using switch statement – Write program to: (a) Print day of week by taking numbers from 1 to 7. (b) Print a student's grade ("A", "B", "C" etc.) by accepting his/her percent marks. (c) Find if a given character is a vowel. (d) Using "if" and "switch" statements – write programs to check whether the triangle is equilateral, isosceles, or scalene.	CO1, CO2, CO3
5	Write Program to: (a) Find sum of digits of a given number. (b) Generate multiplication table up to 10 for numbers 1 to 5. (c) Find Fibonacci series for given number. (d) Write a program to produce the following output:	CO1, CO2, CO3

	<pre> 1 2 3 4 5 6 7 8 9 10 </pre>	
6	Develop a Program to: (a) Sort list of 10 numbers. (b) Perform addition of 3×3 matrix.	CO1, CO2, CO3, CO4
7	Develop Program to: (a) Create a structure called “library” to hold details of a book viz. accession number, title of the book, author name, price of the book, and flag indicating whether the book is issued or not. Fetch some sample data and display the same. (b) Develop and execute C Program to Add Two Distances given in kilometer-meter Using Structures.	CO1, CO2, CO3, CO4
8	Develop Program to demonstrate: (a) Use of all String handling functions (b) Use of few mathematical functions. (c) Use of few other miscellaneous functions. Develop a Program to: (a) Create a function to find the GCD of a given number. Call this function in a program. (b) Find Factorial of given number using recursion.	CO1, CO2, CO3, CO4, CO5
9	(a) Develop a Program to Print values of variables and their addresses. (b) Develop a Program to Find sum of all elements stored in given array using pointers.	CO1, CO2, CO3, CO4, CO6

1. COURSE INFORMATION

Particular	Details
Course Code	2418101 (T2418101 / P2418101 / S2418101)
Course Title	Programming with 'C' (CSE, AIML)
Programme	Diploma in Computer Science & Engineering
Semester	II
Laboratory Instruction	04 Hours/Week
Progressive Lab Assessment (PLA)	20 Marks
End Laboratory Assessment (ELA)	30 Marks

Rationale: The course develops programming logic, algorithms, flowcharts and the ability to implement solutions using the C programming language. It establishes foundation skills in program structure, data handling, control structures, arrays, structures, functions and pointers.

2. COURSE OUTCOMES (COs)

CO	Course Outcome
CO1	Develop flowchart and algorithm to solve problems logically.
CO2	Write simple 'C' programs using arithmetic expressions.
CO3	Develop 'C' programs using control structure.
CO4	Develop 'C' programs using arrays and structures.
CO5	Develop/Use functions in C programs for modular programming approach.
CO6	Develop 'C' programs using pointers.

3. CO-PO ARTICULATION MATRIX

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	1	2	1	-	-	-	-
CO2	1	1	1	1	-	-	-
CO3	1	2	1	1	-	-	-
CO4	1	2	1	1	-	-	-
CO5	1	2	2	1	-	-	-
CO6	1	2	3	1	-	-	-

Legend: 3 = High, 2 = Medium, 1 = Low, - = No Mapping

4. GENERAL LABORATORY INSTRUCTIONS

1. Read the aim, theory and procedure before starting each practical.
2. Prepare the algorithm or program logic before coding.
3. Use meaningful identifiers, proper indentation and suitable comments.
4. Compile the program, remove syntax errors and test with suitable inputs.
5. Record the source code, sample input/output, result and interpretation.
6. Verify boundary and exceptional cases wherever applicable.
7. Follow laboratory discipline and save programs using meaningful filenames.
8. Complete the viva-voce and obtain faculty verification for every practical.

5. PROGRESSIVE LAB ASSESSMENT RUBRIC - 20 MARKS

S.N o.	Category	Criteria of Assessment	Marks
1	Programming and Execution	Implementation of code / algorithm	3

2		Code quality (readability, structure, comments)	3
3		Debugging and testing	2
4	Result and Documentation	Accuracy of output / results	3
5		Correctness of logic & approach	5
6		Completeness of lab record	2
7	Viva-Voce	Conceptual understanding	2
		TOTAL	20

Distribution: Programming and Execution = 8 Marks | Result and Documentation = 10 Marks | Viva-Voce = 2 Marks

6. COMMON PRACTICAL ASSESSMENT RECORD

Practical No.	Date	Programming & Execution (8)	Result & Documentation (10)	Viva-Voce (2)	Total (20)
1					

Faculty Signature: _____

PRACTICAL 1: CONSTANTS, VARIABLES, ARITHMETIC EXPRESSIONS, INCREMENT/DECREMENT AND DATA TYPE CONVERSION

1. Aim

Develop and execute C programs using constants, variables, arithmetic expressions, increment/decrement operators and data type conversion.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

A C program is built using tokens such as keywords, identifiers, constants, operators and punctuators. Variables represent named memory locations whose values may change during program execution, whereas constants represent fixed values. Arithmetic expressions combine operands and operators such as +, -, *, / and %. Increment (++) and decrement (--) modify a variable by one and may be used in prefix or postfix form. Data type conversion allows a value to be converted from one type to another. Implicit conversion is performed automatically by the compiler, while explicit conversion uses a type cast. Correct data types and expression formation are essential for accurate results.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2	LSO 1.1: Write, execute and debug simple C programs. LSO 1.2: Use variables and arithmetic expressions.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Constants, variables and arithmetic expressions

```
#include <stdio.h>
int main() {
    const float PI = 3.14159f;
    int a = 12, b = 5;
    printf("Sum = %d\n", a+b);
    printf("Difference = %d\n", a-b);
    printf("Product = %d\n", a*b);
    printf("Quotient = %d\n", a/b);
    printf("Remainder = %d\n", a%b);
    printf("PI = %.2f\n", PI);
    return 0;
}
```

Output

```
Sum = 17
Difference = 7
Product = 60
Quotient = 2
Remainder = 2
PI = 3.14
```

Program B - Increment and decrement operators

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("Initial x = %d\n", x);
    printf("Post-increment = %d\n", x++);
    printf("After post-increment = %d\n", x);
    printf("Pre-increment = %d\n", ++x);
    printf("Post-decrement = %d\n", x--);
    printf("Final x = %d\n", x);
    return 0;
}
```

Output

```
Initial x = 5
Post-increment = 5
After post-increment = 6
Pre-increment = 7
Post-decrement = 7
Final x = 6
```

Program C - Data type conversion

```
#include <stdio.h>
int main() {
    int a = 5, b = 2;
    float result1 = a / b;
    float result2 = (float)a / b;
    printf("Without explicit cast = %.2f\n", result1);
    printf("With explicit cast = %.2f\n", result2);
}
```

```
return 0;
}
```

Output

Without explicit cast = 2.00
With explicit cast = 2.50

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a variable in C.	✓		
2	State any four arithmetic operators.	✓		
3	Define explicit type conversion.	✓		
4	Differentiate constants and variables.		✓	
5	Explain prefix and postfix increment.		✓	
6	Discuss implicit and explicit conversion.		✓	
7	Apply type casting to obtain 5/2 as 2.5.			✓
8	Create an expression to calculate the remainder.			✓
9	Use increment operator to increase a counter.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 2: INPUT/OUTPUT PROGRAMS: TEMPERATURE CONVERSION, RECTANGLE AND CIRCLE

1. Aim

Use scanf() and printf() to develop programs for temperature conversion and geometrical calculations.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

Standard input and output in C are commonly performed using scanf() and printf(), declared in stdio.h. Format specifiers such as %d, %f, %c and %lf identify the data type to be read or displayed. The address operator (&) is generally used with scanf() so that the function can store input in the corresponding variable. Formula-based programs translate mathematical relationships into arithmetic expressions. Proper use of floating-point types is important in temperature, area, perimeter and circumference calculations because results may contain fractional values.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2	LSO 2.1: Use scanf statement to take user input. LSO 2.2: Use printf statement to print output.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Fahrenheit to Centigrade

```
#include <stdio.h>
int main() {
    float f, c;
```

```
printf("Enter temperature in Fahrenheit: ");
scanf("%f", &f);
c = (f - 32) * 5.0f / 9.0f;
printf("Centigrade = %.2f\n", c);
return 0;
}
```

Output

```
Enter temperature in Fahrenheit: 98.6
Centigrade = 37.00
```

Program B - Area and perimeter of rectangle

```
#include <stdio.h>
int main() {
    float l, b;
    printf("Enter length and breadth: ");
    scanf("%f%f", &l, &b);
    printf("Area = %.2f\n", l*b);
    printf("Perimeter = %.2f\n", 2*(l+b));
    return 0;
}
```

Output

```
Enter length and breadth: 10 5
Area = 50.00
Perimeter = 30.00
```

Program C - Area and circumference of circle

```
#include <stdio.h>
#define PI 3.14159
int main() {
    float r;
    printf("Enter radius: ");
    scanf("%f", &r);
    printf("Area = %.2f\n", PI*r*r);
    printf("Circumference = %.2f\n", 2*PI*r);
    return 0;
}
```

Output

```
Enter radius: 7
Area = 153.94
Circumference = 43.98
```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	State the purpose of scanf().	✓		
2	State the format specifier for float.	✓		
3	State the formula for circumference of a circle.	✓		
4	Explain the role of the address operator in scanf().		✓	
5	Differentiate input and output statements.		✓	
6	Discuss the use of floating-point variables in formula-based programs.		✓	
7	Apply the conversion formula for a given Fahrenheit value.			✓
8	Develop an expression for rectangle perimeter.			✓
9	Select suitable format specifiers for length and breadth.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 3: DECISION MAKING: LEAP YEAR, PALINDROME, GREATEST OF THREE AND VOWEL TEST

1. Aim

Develop C programs using decision-making and two-way branching statements.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

Decision-making statements control program flow according to conditions. Relational operators compare values, while logical operators combine conditions. The if statement executes a block when a condition is true; if-else provides two-way selection; nested if and else-if ladders support multiple related decisions. The conditional operator ?: provides a compact expression for simple two-way selection. A palindrome string reads the same in forward and reverse order. Leap-year testing uses divisibility conditions involving 4, 100 and 400. Careful condition ordering prevents logical errors.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3	LSO 3.1: Write C programs using decision making and two-way branching statements.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Leap year

```
#include <stdio.h>
int main() {
    int y;
```

```
scanf("%d", &y);
if ((y%400==0) || (y%4==0 && y%100!=0))
    printf("Leap year\n");
else
    printf("Not a leap year\n");
return 0;
}
```

Output

2024
Leap year

Program B - String palindrome

```
#include <stdio.h>
#include <string.h>
int main() {
    char s[100]; int i, n, flag=1;
    scanf("%99s", s);
    n = strlen(s);
    for(i=0; i<n/2; i++)
        if(s[i] != s[n-1-i]) { flag=0; break; }
    printf(flag ? "Palindrome\n" : "Not palindrome\n");
    return 0;
}
```

Output

madam
Palindrome

Program C - Greatest of three using conditional operator

```
#include <stdio.h>
int main() {
    int a,b,c,max;
    scanf("%d%d%d",&a,&b,&c);
    max = (a>b) ? ((a>c)?a:c) : ((b>c)?b:c);
    printf("Greatest = %d\n",max);
    return 0;
}
```

Output

12 25 18
Greatest = 25

Program D - Vowel using if-else ladder

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char ch;
    scanf(" %c",&ch);
```

```

ch = tolower(ch);
if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
    printf("Vowel\n");
else
    printf("Not a vowel\n");
return 0;
}

```

Output

```

E
Vowel

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Name the basic decision-making statements in C.	✓		
2	State the leap-year divisibility rule.	✓		
3	Define the conditional operator.	✓		
4	Differentiate if and if-else.		✓	
5	Explain an else-if ladder.		✓	
6	Discuss the logic used for palindrome testing.		✓	
7	Construct a condition for leap-year testing.			✓
8	Apply the conditional operator to find a larger value.			✓
9	Develop a condition to identify vowels.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 4: SWITCH-CASE AND MULTI-WAY BRANCHING

1. Aim

Use switch-case and if statements for multi-way branching problems.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

The switch statement provides multi-way selection based on the value of an integral or character expression. Each case label represents a constant value, break normally terminates the selected case, and default handles unmatched values. Switch is suitable for menu-like choices and discrete values, whereas if-else is more flexible for ranges and compound conditions. Fall-through occurs when break is omitted. Classification problems such as grades and triangle types require careful ranges or comparisons and may use if-else alongside switch where appropriate.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3	LSO 4.2: Write C programs using switch-case for multi-way branching and use if/switch appropriately.

5. Resources Required

S.N o.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Day of week

```
#include <stdio.h>
int main() {
    int d; scanf("%d", &d);
    switch(d) {
```

```
        case 1: puts("Monday"); break; case 2: puts("Tuesday"); break;
        case 3: puts("Wednesday"); break; case 4: puts("Thursday");
break;
        case 5: puts("Friday"); break; case 6: puts("Saturday"); break;
        case 7: puts("Sunday"); break; default: puts("Invalid");
    }
    return 0;
}
```

Output

```
3
Wednesday
```

Program B - Student grade

```
#include <stdio.h>
int main() {
    int m; scanf("%d",&m);
    if(m>=80) puts("A");
    else if(m>=70) puts("B");
    else if(m>=60) puts("C");
    else if(m>=50) puts("D");
    else puts("F");
    return 0;
}
```

Output

```
76
B
```

Program C - Vowel using switch

```
#include <stdio.h>
#include <ctype.h>
int main() {
    char ch; scanf(" %c",&ch); ch=tolower(ch);
    switch(ch) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
puts("Vowel"); break;
        default: puts("Not a vowel");
    }
    return 0;
}
```

Output

```
a
Vowel
```

Program D - Triangle classification

```
#include <stdio.h>
int main() {
    float a,b,c; scanf("%f%f%f",&a,&b,&c);
```

```

    if(a+b<=c || a+c<=b || b+c<=a) puts("Invalid triangle");
    else if(a==b && b==c) puts("Equilateral");
    else if(a==b || b==c || a==c) puts("Isosceles");
    else puts("Scalene");
    return 0;
}

```

Output

```

5 5 8
Isosceles

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	State the purpose of switch.	✓		
2	State the role of break.	✓		
3	Define the default case.	✓		
4	Differentiate switch and if-else ladder.		✓	
5	Explain fall-through in switch.		✓	
6	Discuss the condition for a valid triangle.		✓	
7	Create a switch case for day number 1.			✓
8	Apply conditions to classify an isosceles triangle.			✓
9	Select a suitable branching structure for grade ranges.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 5: LOOPING PROGRAMS: SUM OF DIGITS, MULTIPLICATION TABLES, FIBONACCI SERIES AND PATTERN

1. Aim

Develop iterative C programs using loop statements.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

Loops repeat a statement or block until a termination condition is met. The for loop is convenient when the number of repetitions is known, while and do-while are useful for condition-controlled repetition. A do-while loop executes its body at least once. Loop control depends on initialization, condition checking and update. Digit-processing algorithms commonly use modulus and integer division. Fibonacci generation updates successive terms. Nested loops are useful for tables and patterns. Incorrect update or condition logic can cause infinite loops.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3	LSO 5.1: Write and execute C programs using various loop statements to solve iterative problems.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Sum of digits

```
#include <stdio.h>
int main() {
```

```
int n,sum=0; scanf("%d",&n);
while(n!=0){ sum += n%10; n/=10; }
printf("Sum = %d\n",sum); return 0;
}
```

Output

```
1234
Sum = 10
```

Program B - Multiplication tables 1 to 5

```
#include <stdio.h>
int main() {
    int n,i;
    for(n=1;n<=5;n++){
        printf("\nTable of %d\n",n);
        for(i=1;i<=10;i++) printf("%d x %d = %d\n",n,i,n*i);
    }
    return 0;
}
```

Output

```
Table of 1
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
1 x 9 = 9
1 x 10 = 10

Table of 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20

Table of 3
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
```

```
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
3 x 10 = 30
```

Table of 4

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
```

Table of 5

```
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```

Program C - Fibonacci series

```
#include <stdio.h>
int main() {
    int n,i,a=0,b=1,c; scanf("%d",&n);
    for(i=1;i<=n;i++){ printf("%d ",a); c=a+b; a=b; b=c; }
    return 0;
}
```

Output

```
8
0 1 1 2 3 5 8 13
```

Program D - Number pattern

```
#include <stdio.h>
int main() {
    int i,j,k=1;
    for(i=1;i<=4;i++){
        for(j=1;j<=i;j++) printf("%d\t",k++);
        printf("\n");
    }
}
```

```

    return 0;
}

```

Output

```

1
2      3
4      5      6
7      8      9      10

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S. No.	Viva-Voce Question	R	U	A
1	Name the three common loop statements.	✓		
2	Define an infinite loop.	✓		
3	State the purpose of the modulus operator in digit processing.	✓		
4	Differentiate while and do-while.		✓	
5	Explain nested loops.		✓	
6	Discuss the term update in Fibonacci generation.		✓	
7	Construct a loop to print 1 to 10.			✓
8	Apply modulus and division to process digits.			✓
9	Develop nested loops for a triangular number pattern.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 6: ARRAYS: SORTING AND MATRIX ADDITION

1. Aim

Develop C programs using one-dimensional and two-dimensional arrays.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

An array stores multiple elements of the same data type under one name in contiguous memory locations. Array indexing begins at zero in C. One-dimensional arrays are useful for lists, while two-dimensional arrays represent tables and matrices. Sorting rearranges elements in a chosen order; simple methods repeatedly compare and exchange values. Matrix addition requires equal dimensions and adds corresponding elements. Array bounds must be respected because C does not automatically check invalid indices.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3, CO4	LSO 6.1: Write and execute C programs using one-dimensional arrays. LSO 6.2: Use two-dimensional arrays.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Sort 10 numbers

```
#include <stdio.h>
int main() {
    int a[10], i, j, t;
```

```

for(i=0;i<10;i++) scanf("%d",&a[i]);
for(i=0;i<9;i++)
    for(j=0;j<9-i;j++)
        if(a[j]>a[j+1]){ t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
for(i=0;i<10;i++) printf("%d ",a[i]);
return 0;
}

```

Output

```

9 4 7 1 5 2 10 6 3 8
1 2 3 4 5 6 7 8 9 10

```

Program B - Addition of 3x3 matrices

```

#include <stdio.h>
int main() {
    int a[3][3],b[3][3],c[3][3],i,j;
    for(i=0;i<3;i++) for(j=0;j<3;j++) scanf("%d",&a[i][j]);
    for(i=0;i<3;i++) for(j=0;j<3;j++) scanf("%d",&b[i][j]);
    for(i=0;i<3;i++) for(j=0;j<3;j++) c[i][j]=a[i][j]+b[i][j];
    for(i=0;i<3;i++){ for(j=0;j<3;j++) printf("%d\t",c[i][j]);
printf("\n"); }
    return 0;
}

```

Output

```

1 2 3 4 5 6 7 8 9
9 8 7 6 5 4 3 2 1
10 10 10
10 10 10
10 10 10

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.N o.	Viva-Voce Question	R	U	A
1	Define an array.	✓		
2	State the first valid array index.	✓		
3	Define a two-dimensional array.	✓		
4	Differentiate one-dimensional and two-dimensional arrays.		✓	
5	Explain contiguous storage of array elements.		✓	
6	Discuss the logic of matrix addition.		✓	
7	Declare an integer array of ten elements.			✓
8	Apply comparison and swapping to sort values.			✓
9	Develop an expression to add corresponding matrix elements.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 7: STRUCTURES: LIBRARY RECORD AND ADDITION OF DISTANCES

1. Aim

Use structures to represent related data and solve record-based problems.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

A structure is a user-defined data type that groups variables of different data types under one name. Structure members are declared inside a struct definition and accessed using the dot operator for ordinary structure variables. Structures are suitable for records such as books, students and distances because related fields can be stored together. Structure variables may be initialized, assigned and passed to functions. Addition of compound quantities such as kilometer-meter requires normalization so that the smaller unit remains within its valid range.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3, CO4	LSO 7.1: Write and execute C programs using structures to solve given problems.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Library structure

```
#include <stdio.h>
struct Library {
    int accession;
    char title[50], author[50];
```

```

float price;
int issued;
};
int main() {
    struct Library b;
    printf("Accession: "); scanf("%d",&b.accession);
    printf("Title: "); scanf(" %49[^\n]",b.title);
    printf("Author: "); scanf(" %49[^\n]",b.author);
    printf("Price: "); scanf("%f",&b.price);
    printf("Issued(1/0): "); scanf("%d",&b.issued);
    printf("\n%d | %s | %s | %.2f | %s\n",b.accession,b.title,b.author,b.price,b.issued?"Issued":"Available");
    return 0;
}

```

Output

```

Accession: 101
Title: Programming in ANSI C
Author: E Balagurusamy
Price: 550
Issued(1/0): 0

101 | Programming in ANSI C | E Balagurusamy | 550.00 | Available

```

Program B - Add two distances

```

#include <stdio.h>
struct Distance { int km; float m; };
int main() {
    struct Distance d1,d2,s;
    scanf("%d%f",&d1.km,&d1.m);
    scanf("%d%f",&d2.km,&d2.m);
    s.km=d1.km+d2.km; s.m=d1.m+d2.m;
    if(s.m>=1000){ s.km += (int)(s.m/1000); s.m = (int)s.m % 1000; }
    printf("%d km %.2f m\n",s.km,s.m);
    return 0;
}

```

Output

```

5 750
3 500
9 km 250.00 m

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.

4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a structure in C.	✓		
2	State the operator used to access a structure member.	✓		
3	Name the keyword used to declare a structure.	✓		
4	Differentiate an array and a structure.		✓	
5	Explain the benefit of grouping related fields.		✓	
6	Discuss normalization in kilometer-meter addition.		✓	
7	Declare a structure for a book record.			✓
8	Access the price member of a structure variable b.			✓
9	Apply structure arithmetic to add two distances.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 8: FUNCTIONS: STRING, MATHEMATICAL AND MISCELLANEOUS FUNCTIONS; GCD AND RECURSION

1. Aim

Demonstrate library functions, user-defined functions, GCD calculation and recursion.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

Functions support modular programming by dividing a large problem into smaller reusable units. Library functions are predefined and made available through header files, such as string functions in `string.h` and mathematical functions in `math.h`. A user-defined function includes a return type, name, parameter list and body. Parameters transfer data between calling and called functions. Recursion occurs when a function calls itself and requires a base case to terminate. GCD may be computed efficiently using Euclid's algorithm. Proper prototypes improve type checking and program readability.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3, CO4, CO5	LSO 8.1: Use library functions. LSO 8.2: Create and use user-defined functions.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - String functions

```
#include <stdio.h>
#include <string.h>
int main() {
```

```

char a[100]="Hello", b[50]=" C";
printf("Length = %zu\n",strlen(a));
strcat(a,b);
printf("Concatenated = %s\n",a);
strcpy(b,"Hello");
printf("Comparison = %d\n",strcmp(a,b));
return 0;
}

```

Output

```

Length = 5
Concatenated = Hello C
Comparison = 32

```

Program B - Mathematical functions

```

#include <stdio.h>
#include <math.h>
int main() {
    double x=25.0;
    printf("sqrt = %.2f\n",sqrt(x));
    printf("pow = %.2f\n",pow(2,5));
    printf("ceil = %.2f\n",ceil(4.2));
    printf("floor = %.2f\n",floor(4.8));
    return 0;
}

```

Output

```

sqrt = 5.00
pow = 32.00
ceil = 5.00
floor = 4.00

```

Program C - GCD using function

```

#include <stdio.h>
int gcd(int a,int b){ while(b){ int r=a%b; a=b; b=r; } return a; }
int main(){ int a,b; scanf("%d%d",&a,&b); printf("GCD = %d\n",gcd(a,b)); return 0; }

```

Output

```

24 18
GCD = 6

```

Program D - Factorial using recursion

```

#include <stdio.h>
long long fact(int n){ if(n<=1) return 1; return n*fact(n-1); }
int main(){ int n; scanf("%d",&n); printf("Factorial = %lld\n",fact(n)); return 0; }

```

Output

```

5
Factorial = 120

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a function prototype.	✓		
2	Name the header file for string functions.	✓		
3	Define recursion.	✓		
4	Differentiate library and user-defined functions.		✓	
5	Explain the purpose of a base case.		✓	
6	Discuss modular programming using functions.		✓	
7	Create a function call for gcd(24,18).			✓
8	Apply recursion to factorial of 5.			✓
9	Select a suitable library function to calculate square root.			✓

Legend : R = Remember | U = Understanding | A = Apply

PRACTICAL 9: POINTERS: ADDRESSES AND SUM OF ARRAY ELEMENTS

1. Aim

Develop C programs using pointers to access memory addresses and array elements.

2. Practical Significance

This practical strengthens programming logic and implementation skills related to the corresponding SBTE laboratory outcome. It emphasizes correct program construction, compilation, testing, interpretation of output and disciplined documentation.

3. Relevant Theory

A pointer is a variable that stores the address of another object. The address operator (&) obtains an address, while the dereference operator (*) accesses the value stored at the referenced address. A pointer must be declared with an appropriate base type and should be initialized before dereferencing. Array names are closely related to pointers because the array name represents the address of its first element in most expressions. Pointer arithmetic moves according to the size of the pointed data type. Pointers support efficient array traversal, parameter passing and dynamic data structures.

4. Relevant CO(s) and LSO(s)

Relevant CO(s)	Laboratory Session Outcome(s)
CO1, CO2, CO3, CO4, CO6	LSO 9.1: Use pointers to manipulate data by accessing computer memory.

5. Resources Required

S.No.	Resource	Specification
1	Computer System	Basic computer system
2	C Compiler / IDE	Any ANSI C compiler such as GCC / Code::Blocks / Dev-C++
3	Operating System	Windows / Linux

6. Suggested Procedure

1. Study the problem statement and identify required inputs and outputs.
2. Prepare the logic/algorithm and select suitable C constructs.
3. Write the source program using meaningful identifiers and indentation.
4. Compile the program and correct syntax or semantic errors.
5. Execute the program with suitable test data.
6. Verify the output and record observations and result.

7. Programs / Implementation

Program A - Print values and addresses

```
#include <stdio.h>
int main() {
    int a=10; float b=2.5f; char c='X';
    printf("a=%d address=%p\n",a, (void*)&a);
}
```

```

printf("b=%.2f address=%p\n",b, (void*)&b);
printf("c=%c address=%p\n",c, (void*)&c);
return 0;
}

```

Output

```

a=10 address=0XXXXXXXXX
b=2.50 address=0XXXXXXXXX
c=X address=0XXXXXXXXX

```

Note: memory addresses vary by system/execution.

Program B - Sum of array elements using pointers

```

#include <stdio.h>
int main() {
    int a[5],i,sum=0,*p=a;
    for(i=0;i<5;i++) scanf("%d",p+i);
    for(i=0;i<5;i++) sum += *(p+i);
    printf("Sum = %d\n",sum);
    return 0;
}

```

Output

```

10 20 30 40 50
Sum = 150

```

8. Result

The specified C programs were written, compiled, executed and tested successfully. The obtained results were verified against the intended program logic.

9. Interpretation

The practical demonstrates correct application of the relevant C programming concepts. Testing with suitable inputs helps verify program correctness and reveals logical or boundary-case errors.

10. Cautions

1. Use correct data types and format specifiers.
2. Initialize variables before use.
3. Maintain correct syntax, braces and statement termination.
4. Test the program with more than one input set.
5. Avoid accessing invalid array indices or uninitialized pointers where applicable.

11. Viva-Voce Questions

S.No.	Viva-Voce Question	R	U	A
1	Define a pointer.	✓		
2	State the purpose of the address operator.	✓		
3	State the purpose of the dereference operator.	✓		
4	Explain the relationship between arrays and pointers.		✓	
5	Differentiate an address and a stored value.		✓	

6	Discuss pointer arithmetic for integer arrays.		✓	
7	Declare a pointer to an integer.			✓
8	Apply pointer notation to access the third array element.			✓
9	Use a pointer to calculate the sum of array elements.			✓

Legend : R = Remember | U = Understanding | A = Apply

SUGGESTED LEARNING RESOURCES

S.No.	Title	Author(s)	Publisher / Edition
1	Programming in ANSI C	E. Balagurusamy	McGraw Hill Education, New Delhi, 2019
2	The C Programming Language	Brian W. Kernighan, Dennis Ritchie	PHI Learning Private Limited
3	Let Us C	Yashavant Kanetkar	BPB Publications, New Delhi, 2020